

제3회 ITDA 연합학술제 본선

분석·설계 및 도메인 기획 보고서

상품 후면 라벨 사진에서 소비기한 추출

[DScover]_카피바라

성균관대학교 DScover 12기

2026. 10. 01.

목 차

1. 개괄

1-1 핵심 접근 방식 · 1-2 주요 결과

2. 데이터 구성 및 라벨링 전략

2-1 사용 데이터 · 2-2 라벨링 규칙과 절차 · 2-3 전처리

3. 아키텍처 설계 논리

3-1 구조도 · 3-2 모듈별 설계 의도 · 3-3 후처리 규칙

4. 모델 선택 근거 및 성능 평가

4-1 비교·선택 · 4-2 정확도(검증 방법 포함) · 4-3 오류 분석과 개선

5. 속도·비용 검증

5-1 CPU 추론 속도 · 5-2 속도·오탐 최적화 · 5-3 학습 자원

6. 도메인 활용 및 운영 기획

6-1 적용 도메인 · 6-2 운영 아키텍처 · 6-3 운영 비용 · 6-4 ROI

7. 한계와 보완 계획

8. 추가 참고 자료

1. 개괄

1-1. 핵심 접근 방식 요약

실패한 이미지만 다음 단계로 넘기는 계단형(cascade) 파이프라인이다. 쉬운 이미지는 가장 싼 수단으로 처리하고, 실패한 것에만 더 비싼 수단을 순서대로 투입한다. 각 단계는 **앞 단계가 실패한 이미지에만** 실행되므로, 새 수단을 추가해도 기존 정답이 깨지는 일이 구조적으로 발생하지 않는다. 회귀 위험이 0이기 때문에 공격적으로 실험할 수 있었고, 실제로 수단 12가지를 시험해 5가지만 채택했다.

두 번째 축은 **판정 규율**이다. 제공 데이터에 정답이 없어 828장을 직접 라벨링한 뒤 다섯 집합으로 나누고, 한 번 판정에 쓴 집합은 폐기했다. 채택 여부는 '개선처럼 보이는가'가 아니라 '설계에 쓰지 않은 표본에서 효과가 재현되는가'로 정했다.

1-2. 주요 결과 요약

구분	측정값	조건
날짜 추출 정확도	92.3%	홀드아웃4+5 합산 248장 · 설계에 미사용 · 95% CI 90.2~94.0%
완전일치(3필드 동시)	89.5%	동일 표본
미검출	10 / 248	아무 값도 내지 못한 경우
오탐	0건	틀린 값을 확정 출력한 경우
CPU 추론 속도	1.01초/장	단일 프로세스 순차 · 채점과 동일한 500장 실측 = 503초
노트북 완주 검증	459초	nbconvert --execute 500장 · 모델 로딩 포함 백시계
제한 대비	19~21%	제한 2,400초 / 500장 (약 5배 여유)
학습 비용	0원	GPU·클라우드·외부 API 미사용. 전 과정 로컬 CPU

도메인 기획 한 줄 요약 — 오탐 0건을 보장하는 '무응답 우선' 설계로, 소매·물류의 유통기한 검품을 표본검사에서 전수검사로 전환한다.

본 보고서의 모든 성능·속도 수치는 실측이며 재현 스크립트가 eda/에 119건 있다. 추정이 포함된 항목(6-3 운영 비용, 6-4 ROI)은 해당 절에 가정을 명시했다.

2. 데이터 구성 및 라벨링 전략

2-1. 사용 데이터

제공 데이터 3,352장만 사용했고 **외부 데이터를 추가 수집하지 않았다**. 다만 제공 데이터에는 **정답 라벨이 전혀 없어** 자체 라벨링이 선행 과제였다.

용도	수량	내용
정답 라벨링	828장	직접 판독한 year/month/day (전체의 24.7%)
실패 목록 집계	3,164장	전량 실행해 미검출·오답 분포 확인 (EDA 69)
YOLO 학습	237장 / 238박스	약지도 자동 생성 (2-2 참고)
도트 판독기 학습	합성 40,000장	실사진 미사용 · 폰트에 점 격자를 씌워 생성

2-2. 라벨링 규칙 정의 및 수행 절차

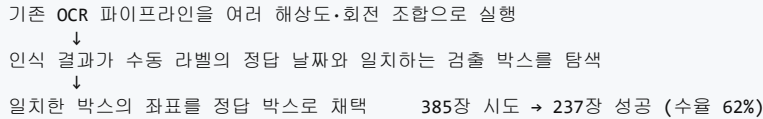
(1) 수동 라벨링 — 정답 828장

브라우저 기반 라벨링 도구를 직접 제작해 이미지를 확대하며 판독했다. 판독 규칙은 다음과 같이 고정했다.

- 소비기한·유통기한이 우선. 제조일자만 있으면 그 값을 쓰지 않고 NONE 처리한다.
- 연·월·일 중 일부만 인쇄된 경우 읽은 필드만 채우고 나머지는 NONE (부분점수 대응).

- 검수** — 무작위 30장을 다시 판독해 대조한 결과 30/30 일치했다. 다만 **0건 오류의 95% 신뢰상한은 약 10%**이므로 강한 보증은 아니며, 이 점을 7장 한계에 명시한다.

YOLO 학습에는 날짜의 '값'이 아니라 '위치(좌표)'가 필요한데, 수동 라벨에는 값만 있다. 그래서 약지도(weak supervision) 방식으로 좌표를 자동 생성했다.



2-3. 전처리 내용

전처리	적용 위치	채택 여부와 근거
긴 변 1024px 축소	전 이미지 공통	채택. 3200px까지 올려도 검출률 불변, 비용만 증가
정사각 패딩	검출기 입력	채택. 종횡비 왜곡 없이 640 정사각 입력에 맞춤
adaptive 이진화	2칸 (실패분)	채택. 조명 얼룩이 있는 라벨에서 회수
CLAHE 대비 보정	4칸 (실패분)	채택. 저대비 인쇄 3장 회수
샤프닝/디블러	—	기각. 오답이 오히려 더 선명했다(선명도 중앙값 458 대 412)
기타 8종	—	기각. 23장 중 3장만 회수되어 상시 적용 비용이 이득을 넘음

3. 아키텍처 설계 논리

입력 이미지 1장 → imread_any() → downscale(긴 변 1024px)

※ 각 칸은 '앞 칸이 실패한 이미지'에만 실행된다 (괄호 = 도달 비율)

1칸	RapidOCR PP-OCRv6 det small @1024px	80%	여기서 종료
2칸	adaptive 이진화 후 재시도	2%	
3칸	det medium (대형 검출기)	5%	
4칸	CLAHE 대비 보정 + det medium	1%	
5칸	인식기 교체 - 한국어 모델	1%	
6칸	인식기 교체 - 영문 모델	1%	
7칸	도트 인쇄 전용 판독기 (직접 학습 CRNN+CTC)	1%	
보조	직접 학습 YOLOv8n 검출 → PP-OCR 인식 (최후수단)	1%	

각 칸이 만들어 낸 텍스트 조각

날짜 규칙 엔진 앵커 점수화 → 형식 정규화 → 연·월·일 분해

year / month / day (확신이 없으면 NONE - 틀린 값보다 무응답)

3-2. 모듈별 설계 의도 및 기술 선택 이유

모듈	선택	왜 이렇게 설계했는가
전체 구조	계단형	새 수단이 기존 정답을 깨뜨릴 수 없어야 실험을 빠르게 할 수 있다. 모든 칸을 항상 실행하는 구조는 회귀 위험 때문에 시도 횟수가 줄어든다.
주력 엔진	RapidOCR PP-OCRv6	ONNX Runtime 기반이라 torch 의존이 없다. torch가 환경에 있으면 OpenMP 충돌로 2.4배 느려지는 것을 실측했다(EDA 52).
검출 해상도	1024px 고정	3200px까지 올려도 검출률이 오르지 않고 비용만 증가(EDA 5).
인식기 2종	한국어 + 영문	기본 인식기가 중국어 모델이라 한글을 읽지 못한다. 유통2023.01.05까지처럼 한글이 낱자에 붙으면 낱자까지 깨진다(EDA 101).
도트 판독기	CRNN + CTC	글자를 잘라 분류하는 방식은 8글자가 모두 정확히 분할돼야 하므로 실사용 정답 0장이었다. 분할을 없애는 CTC로 전환해 해결(EDA 110→111).
낱자 검출	YOLOv8n → ONNX	보조 경로 전용. ultralytics/torch를 제출물에서 제거하기 위해 ONNX로 내보내 추론한다(EDA 54~55).

3-3. 후처리 규칙 설계

(1) 소비기한과 다른 숫자의 구분

후면 라벨에는 낱자 형태의 숫자가 여러 개 있다(제조일자·포장일자·영양성분표·바코드·중량). 주변 문맥에 점수를 주어 가려낸다.

앵커(주변 단어)	점수	의도
까지	+15	기한의 종점을 직접 지시하는 가장 강한 신호
소비기한 / 유통기한 / 품질유지기한	+10	정답 앵커
부터	-8	기간의 시작점 → 소비기한이 아님
제조일자 / 생산일자 / 포장일자 / MFG	-12	오답 앵커

- 동점이면 더 늦은 낱자를 택한다. 제조일과 소비기한이 함께 인쇄된 경우 소비기한이 항상 뒤였다.
- 영양성분표의 숫자는 낱자 형식 자체를 만족하지 않아 형식 단계에서 걸러진다.
- 시각(11:30)이 낱자의 '일'로 읽히는 것을 막기 위해 패턴에 부정 전방탐색을 넣었다.

(2) 낱자 정규화

12가지 표기 패턴을 정규식으로 처리한다(YYYY.MM.DD, YY.MM.DD, YYYYMMDD, DD MON YYYY, MON DD YY, YYYYMM.DD, 한글 표기 등). 핵심 판단은 2자리 연도의 순서였다. 26.08.13은 한국식이면 2026-08-13, 유럽식이면 2013-08-26이다. 데이터에서 직접 세어 **한국식 가정 91% 대 유럽식 가정 52%**를 확인하고 한국식을 기본값으로 정했다. 이 한 줄이 해당 구역 정확도를 52%에서 91%로 올렸다(EDA 81).

(3) 미인식 NONE 처리 기준

- 앵커 점수 합계가 음수이면 값을 내지 않고 NONE을 출력한다. 제조일자만 인쇄된 이미지가 여기 해당한다.
- 연·월·일은 독립 채점되므로, 일부 필드만 읽힌 경우 읽은 필드는 채우고 나머지만 NONE으로 둔다.
- 계단 중간에 나온 부분 결과는 버리지 않고 보관하다가, 뒤 칸이 **채운 필드 수를 늘릴 때만** 교체한다.

마지막 규칙이 없으면 도트 판독기가 부분 결과(NONE-08-25)를 엉뚱한 값으로 덮어써 이미 맞힌 2개 필드까지 잃는다. 실제로 홀드아웃4가 93.0%→92.8%로 떨어져 발견했다.

4. 모델 선택 근거 및 성능 평가

4-1. 비교한 모델·대안과 최종 선택 근거

비교 대상	결과	판정
PaddleOCR (PP-OCRv3/v4)	예선 주력. 한글 인식은 되나 속도·의존성 부담	보조 경로로 강등
RapidOCR (PP-OCRv6)	정확도·속도 모두 우위 · ONNX 단독 실행	주력 채택
EasyOCR	결합해도 이득 없음 (torch 의존으로 2.4배 저하)	기각
검출기 대형화(det server)	이득 없음, 모델 84MB·비용만 증가	기각
입력 1600px 재시도 칸	오염 표본 +0.6%p → 깨끗한 표본 0장	기각
8칸 전부 실행 후 다수결	93.6% → 93.4%	기각
YOLO 도트 파인튜닝(v2)	mAP50 0.80이나 기존 정답 6장 상실	기각
검출 문턱 0.25 → 0.05	검출률 4%→65%인데 정확도 92.3%→92.2%	기각
인식기 칸(한국어·영문)	개선을 재현 · 메커니즘 특정됨	채택
도트 전용 판독기(CTC)	2장 회수 · 오탐 0	채택

기각한 것이 채택한 것보다 많다. 채택 기준을 '오염되지 않은 표본에서 효과가 재현되는 가'로 고정했기 때문이다.

4-2. 날짜 추출 정확도 결과

(1) 검증 방법

자체 라벨 828장을 다섯 집합으로 나누고, 설계·튜닝에 쓴 집합은 성능 근거에서 배제했다. 최종 수치는 두 개의 미사용 집합을 합쳐 산출했다.

집합	수량	용도	오염 여부
튜닝셋	137	규칙 설계·파라미터 결정	오염
홀드아웃1~3	443	중간 판정 후 오답 검토	오염
홀드아웃4	148	본선 최종 판정	미오염
홀드아웃5	100	인식기 칸 채택 판정	미오염

오답을 한 장씩 검토한 표본은 더 이상 성능의 근거가 되지 못한다. 그 **편향의 크기를 실제로 측정했다** — 검토에 사용한 580장에서는 94.1%가 나오지만 미사용 표본에서는 92.3%로, **1.8%p가 낙관 편향**이다. 보고에는 92.3%만 사용한다.

(2) 수치

표본	필드 정확도	year	month	day	완전일치	미검출	오탐
홀드아웃4 (148장)	92.8%	93.2%	92.6%	92.6%	90.5%	6	0
홀드아웃5 (100장)	91.7%	92.0%	93.0%	90.0%	88.0%	4	0
합산 (248장)	92.3%	92.7%	92.7%	91.5%	89.5%	10	0

95% 신뢰구간 90.2~94.0%. **예선 제출 구성을 동일 표본에서 재측정하면 88.5%**이므로 본선 라운드에서 **+4.3%p** 개선했다(약화 0장, McNemar p=0.0156).

4-3. 오류 사례 분석 및 개선 과정

이 절을 가장 중요하게 생각한다. 병목을 두 차례 잘못 지목했고, 두 번 다 측정으로 바로잡았기 때문이다.

(1) 첫 번째 오진 — "점선(도트) 인쇄가 문제다"

미검출 이미지를 육안으로 보니 도트 매트릭스 인쇄가 두드러졌다. 전용 판독기를 만들고 나서야 전체에서 도트 그룹의 성능을 측정했다.

그룹	수량	필드 정확도
도트 인쇄	137장	97.0%
일반 인쇄	498장	95.9%

도트가 오히려 더 잘 읽혔다. 도트는 공장 잉크젯이라 글자가 크고 간격이 균일하다. 실제로 어려운 것은 작고 구겨진 인쇄였다. **육안으로 눈에 띄는 것과 원인은 다르다.**

(2) 두 번째 오진 — "검출기가 낱자 위치를 못 잡는 것이 병목이다"

그룹	수량	필드 정확도	미검출률
YOLO가 위치를 잡음	635장	96.1%	1.1%
YOLO가 못 잡음	193장	85.8%	7.8%

10.2%p 차이라 검출이 병목으로 보였다. 원인을 찾기 위해 낱자 영역의 크기·대비·위치를 두 그룹에서 비교했으나 **어느 지표도 차이를 설명하지 못했다**(높이 0.90배 · 대비 1.05배 · 가장자리 0.91배).

지표가 동일한데 한쪽만 실패한다면 이미지가 어려운 것이 아니다. 검출 문턱(confidence)만 0.25에서 0.02로 낮추자 **못 잡던 193장 중 80%가 검출됐다**. 학습 데이터가 238장뿐이라 모델이 전반적으로 확신이 낮았던 것이다(2-2의 편향). 그런데 문턱을 낮춰 끝단 정확도를 다시 측정한 결과는 다음과 같았다.

표본	기존 (conf 0.25)	변경 (conf 0.05)
홀드아웃4	92.8%	93.2%
홀드아웃5	91.7%	90.7%
합산 248장	92.3%	92.2%

검출률이 30배가 됐는데 정확도는 744필드 중 1개 차이였고, **두 표본에서 부호가 같았다**(홀드아웃4 +2필드, 홀드아웃5 -3필드). 한쪽만 보았다면 "+0.4%p 개선"으로 채택했을 것이다. 기각했다.

이 기각에서 얻은 결론 — 앞의 10.2%p 차이는 상관을 인과로 읽은 것이었다. 검출기가 못 잡는 이미지는 애초에 작고 흐리고 기울어져 있어 인식하기도 어렵다. 검출 실패가 성능을 떨어뜨린 것이 아니라 둘 다 같은 원인의 결과였다. 검출을 강제로 성공시켰는데 결과가 움직이지 않은 것이 그 증거다. **하위 그룹의 성능 차이는 그 자체로 원인을 지목하지 않는다. 원인이라 주장하려면 그 변수를 직접 조작해 결과가 따라오는지 확인해야 한다.**

(3) 남은 오류의 성격

- 규칙으로 해결 가능한 것은 51장 중 2장뿐이다. 대부분 OCR이 글자 자체를 만들어내지 못한다.
- 계단은 온전한 낱자가 나오면 멈추므로 그 값이 틀려도 중단된다. 전 칸 실행 후 다수결은 오히려 더 나뉘었다(93.6→93.4%).
- 오라클 상한(정답을 아는 사람이 8칸 결과 중 최선을 고른다고 가정) 측정값은 94.9%로, 현재 구조의 천장이 멀지 않다.

5. 속도·비용 검증

5-1. CPU 추론 속도 측정

항목	내용
측정 환경	AMD Ryzen 5 7500F · Windows 11 · Python 3.11 · onnxruntime 1.19.2 (CPU) · GPU 미사용
실행 방식	단일 프로세스 순차 루프 (채점 노트북과 동일 조건, 병렬 없음)
측정 규모	채점과 동일한 500장을 그대로 실행 (환산·표본 추정 없음)
이미지당 평균	1.01초 · 중앙값 0.39초 · 상위 10% 2.09초 · 상위 1% 11.10초 · 최대 17.71초
500장 소요	503초 (추론 루프) / 459초 (nbconvert --execute 노트북 완주, 모델 로딩 포함 벽 시계)
제한 대비	19~21% (제한 2,400초, 약 5배 여유)

중앙값이 평균의 40%에 불과하다. 80%가 1칸에서 끝나고 소수가 계단을 끝까지 오르며 시간을 쓰는 꼬리가 무거운 분포다. 이 때문에 표본이 작으면 평균이 낙관적으로 나온다 — 실제로 120장 표본에서는 0.81초, 150장에서는 0.91초가 나왔다. 그래서 속도는 반드시 채점과 같은 500장으로 측정하고, 보고에는 더 느린 쪽을 쓴다.

5-2. 속도 및 오탐 최적화 전략

(1) 속도 — 비용을 실패분에만 지출

계단 구조의 부수 효과가 곧 속도 전략이다. 80%가 1칸에서 종료되므로 무거운 수단(대형 검출기·인식기 교체·도트 판독기)의 비용이 전체 평균에 거의 반영되지 않는다.

구성	장당	500장	제한 대비
torch 포함 환경 (참고)	5.88초	2,939초	122% (초과)
보조 경로 단독	2.19초	1,096초	46%
1차 예선 제출 구성	0.53초	267초	11%
본선 최종 (계단 7칸 + 보조)	1.01초	503초	21%

가장 큰 속도 저하는 torch가 설치된 환경에서 발생했다(OpenMP 스레드 충돌, 2.4배). 제출물에서 torch-ultralytics 의존을 완전히 제거하고 ONNX Runtime만 사용한다. requirements.txt에 두 패키지를 넣지 않는 것이 설계 제약이다.

(2) 오탐 — 확신이 없으면 출력하지 않는다

- 앵커 점수 합계가 음수면 NONE을 출력한다. 제조일자만 인쇄된 이미지에서 오탐을 차단한다.
- '미래 날짜만 유효' 같은 상식 필터는 넣지 않았다. 정답에 2020~2022년 과거 날짜가 다수 있어 오히려 정답을 지운다.
- 부분 결과는 '채운 필드 수가 늘어날 때만' 교체해 이미 맞힌 필드를 잃지 않게 한다.

결과적으로 미사용 표본 248장에서 오탐 0건이다. 미검출 10장은 모두 '값을 내지 않은' 경우로, 잘못된 값을 내보낸 사례가 아니다.

5-3. 학습 자원 사용 내역

유료 GPU·클라우드·외부 API를 일절 사용하지 않았다. 학습·추론 전 과정이 로컬 CPU에서 수행되었으며 직접 비용은 0원이다.

학습 대상	환경	설정	소요	결과	채택
YOLOv8n 낱자 검출	로컬 CPU	120 epoch · 640px · 237장	2,824초	mAP50 0.82	채택
도트 판독기 (CRNN+CTC)	로컬 CPU	합성 40,000장 · 입력 32×192	약 1시간	검증 완전일치 79.4%	채택
YOLO 도트 파인튜닝 v2	로컬 CPU	40 epoch 이어학습	2.4시간	mAP50 0.80	기각
도트 글자 분류기	로컬 CPU	합성 학습	약 30분	검증 96.6% / 실사용 0장	기각

총 학습 시간 약 4.5시간(기각된 실험 포함), 전량 로컬 CPU. 사전학습 가중치는 RapidOCR 공개 모델(PP-OCRv6)을 그대로 사용했다.

6. 도메인 활용 및 운영 기획

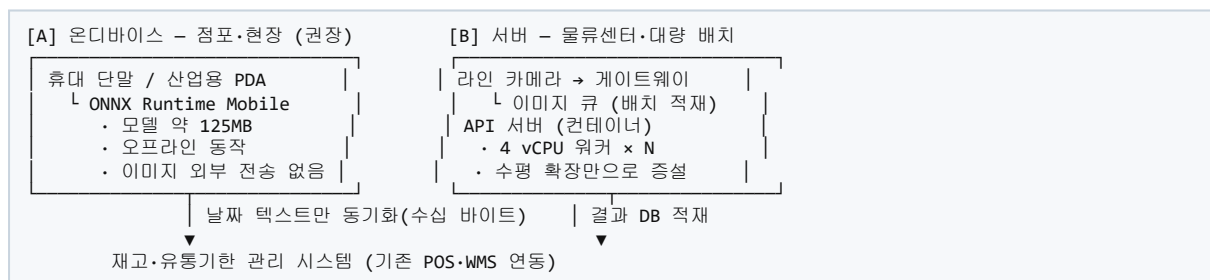
6-1. 적용 도메인 및 도입 대상

핵심 도입 논리는 '표본검사 → 전수검사'다. 유통기한 확인은 지금도 사람이 눈으로 하며, 인건비 때문에 표본검사로 운영된다. 장당 1초·오타 0건이면 전수검사가 성립한다.

도입 대상	현재 업무	적용 효과
편의점·SSM 본부	점포 직원이 진열대를 순회하며 임박 상품 육안 확인	입고 시 후면 촬영으로 기한 자동 등록 → 임박 알림 자동화
식품 물류센터	입고 검수 시 파렛트당 표본 추출 검사	라인 카메라로 전수 검사, 기한 역전(선입선출 위반) 자동 적발
이커머스 신선식품	출고 직전 유통기한 수기 확인	출고 검수 자동화 및 고객 클레임 근거 자료 자동 생성
식자재 유통·급식	납품 검수 시 기한 확인 후 수기 기록	검수 기록 자동 생성 (HACCP 문서화 부담 경감)
시각장애인 보조 앱	포장지의 작은 낱자 판독 불가	촬영 후 음성 안내. 저대비·소형 인쇄 대응이 계단 구조의 강점과 일치

6-2. 운영 아키텍처 설계

파이프라인이 CPU 전용·ONNX 단일 런타임이라는 점이 배포 선택지를 넓힌다. 용도에 따라 두 형태로 운영한다.



- **[A] 온디바이스** — GPU가 필요 없고 오프라인에서 동작한다. 이미지가 단말을 벗어나지 않아 개인정보·영업정보 이슈가 없다. 점포 단위 도입에 적합하다.
- **[B] 서버** — 처리량이 큰 경우, GPU 인스턴스가 필요 없어 일반 CPU 인스턴스로 구성되며, 부하 증가는 워커 수평 확장으로 대응한다.
- **공통** — 미검출(NONE) 건은 사람 확인 큐로 보낸다. **오타 0건 설계이므로 이 큐만 처리하면 되고, 시스템이 잘못된 값을 조용히 흘려보내지 않는다.**

6-3. 예상 운영 비용 산출

※ 아래는 측정값이 아니라 추정이다. 실측 근거는 '장당 1.01초(CPU)'와 'GPU 불요' 두 가지이며, 단가·물량은 공개 클라우드 요금과 일반적인 점포 운영 규모를 가정한 값이다. 실도입 시 재산정이 필요하다.

[A] 온디바이스 (편의점 1개 점포)	가정	비용
단말	기존 점포 PDA·스마트폰 활용	추가 0원
연산	장당 1.01초 · 하루 입고 300개 → 약 5분/일	0원 (단말 유휴)
통신	날짜 텍스트만 전송 (건당 수십 바이트)	사실상 0원
점포당 월 운영비	초기 앱 배포 비용만 발생	0원

[B] 서버 (물류센터 · 일 10만 장)	산출	월 비용(추정)
필요 연산량	100,000장 × 1.01초 = 28.1 CPU-시간/일	—
워커 구성	4 vCPU 인스턴스 2대 상시 (피크 여유 포함)	약 20만 원
스토리지	원본 미보관, 결과 텍스트만 적재	약 2만 원
합계		월 약 22만 원

핵심은 **GPU 인스턴스가 필요 없다는 점**이다. 동급 GPU 인스턴스를 상시 운용하면 월 수백만 원 단위가 되므로, CPU 전용 설계가 곧 운영비 설계다.

6-4. 비즈니스 손실 절감 효과 및 ROI

※ 절감 효과도 추정이다. '장당 1.01초'와 '오탐 0건'만 실측값이고, 점포 규모·폐기율·인건비는 가정이다. 가정을 바꿔 계산할 수 있도록 근거를 함께 적는다.

인건비 절감	수작업	자동화	차이
1건 확인 시간	약 5초 (집기·판독·기록)	1.01초 (촬영 포함 약 2초)	약 60% 단축
일 300건 기준	25분	10분	15분/일
점포 월 환산	12.5시간	5시간	7.5시간
인건비 환산(1만원/시)			월 약 7.5만 원

더 큰 효과는 인건비가 아니라 **폐기 손실**에 있다. 표본검사에서는 임박 상품을 놓쳐 폐기로 이어지지만, 전수 검사가 되면 할인 판매 시점을 앞당길 수 있다. 편의점 점포 월매출 5,000만 원 · 폐기율 3%(150만 원) 가정에서, 조기 발견으로 폐기의 20%를 할인 판매로 전환하면 **월 약 30만 원**이 회수된다.

ROI	월 효과(점포당, 추정)
인건비 절감	약 7.5만 원
폐기 손실 절감	약 30만 원
합계 편익	약 37.5만 원
운영 비용	0원 (온디바이스, 기존 단말 활용)
순효과	약 37.5만 원 / 월 / 점포

초기 개발·배포 비용을 3,000만 원으로 가정하면, **100개 점포 도입 시 약 0.8개월**만에 회수된다. 운영비가 사실상 0에 수렴하는 CPU·온디바이스 구조이기 때문에 도입 점포 수가 늘수록 회수 기간이 짧아진다. 다만 위 수치는 가정에 기반한 추정이며, 실제 도입 전 파일럿 점포에서 폐기율과 검수 시간을 실측해 재산정해야 한다.

7. 한계와 보완 계획

한계	내용	보완 계획
표본 오차	92.3%의 95% 신뢰구간이 90.2~94.0%다. 248장으로는 소수점 한 자리를 다루는 비교가 불가능하다	라벨 확대. 다만 판정용 표본은 한 번 쓰면 폐기해야 하므로 지속 증설이 필요
라벨의 신뢰도	정답을 자체 제작했다. 무작위 30장 재판독에서 30/30 일치했으나 0건 오류의 95% 신뢰상한은 약 10%	교차 판독 인원을 늘려 신뢰상한을 낮춤
판단이 갈리는 라벨	수입품 2자리 연도, 제조일자만 인쇄된 경우, 월·일만 인쇄된 경우	판단 근거를 파일로 남겼으며, 기준 합의 시 재라벨링 가능
인식 자체의 한계	남은 오답의 대부분은 OCR이 글자를 만들어내지 못하는 경우다. 규칙으로 해결 가능한 것은 51장 중 2장	실제 저대비·소형 인쇄 이미지를 수집해 인식기를 파인튜닝 (본선 이후 과제)
첫 성공에서 멈추는 구조	온전한 날짜가 나오면 틀려도 중단한다. 오라클 상한 94.9% 대비 손실분	다수결은 이미 기각(93.6→93.4%). 신뢰도 기반 재검증 방식을 검토
검출기 학습 데이터 편향	약지도로 모은 238박스는 OCR이 이미 성공한 사례뿐이라 어려운 사례가 학습에 없다	검출 개선의 상한이 0으로 추정되어(4-3) 우선순위를 낮춤. 인식기 개선이 선행되어야 의미가 생김
환경 의존 결함	PaddleOCR의 C++ 백엔드가 비ASCII 경로 의 모델을 열지 못하고 조용히 실패한다. 한글 폴더에 두면 보조 경로가 에러 없이 꺼진다	경로가 비ASCII면 ASCII 임시 폴더로 복사해 넘기도록 수정 완료. 248장 재측정에서 예측 0장 불일치 확인
채점 환경 차이	측정은 Ryzen 5 7500F. 채점 CPU가 더 느릴 수 있음	제한의 21%로 약 5배 여유가 있어 2배 느린 환경에서도 42% 수준

8. 추가 참고 자료

자료	내용
METHOD.md	설계 근거 전문. 모든 수치에 대응하는 실험 번호가 붙어 있다
JOURNAL.md	실험 일지. 틀렸던 판단과 그 정정 과정을 포함한다 (병목 오진 2회, 속도 오보 3회)
eda/ · eda/out/	실험 스크립트 119건과 리포트. 스크립트와 결과가 1:1 대응
custom_data/	자체 제작 라벨 828장과 라벨링 도구 (원본 이미지는 재배포하지 않음)
logs/	새 가상환경 재현성 검증 로그 (verify_fresh.ps1 출력)
run_local.py	가중치 점검·보조 엔진 복구·실행·정답 대조를 한 번에 수행하는 점검 스크립트

저장소에는 기각된 실험도 모두 남겨 두었다(src/rejected_rungs.py). 어떤 판단이 왜 틀렸는지가 남아 있어야 같은 실수를 반복하지 않는다고 생각하기 때문이다.